Measurement-Driven Selective Abstraction for Performance-Portable HPC Software

Comprehensive Experimental Design Document

Research Framework & Implementation Plan

Executive Summary

This document presents a comprehensive experimental design for investigating measurement-driven selective abstraction in high-performance computing (HPC) software. The research addresses a critical gap in exascale programming: the lack of systematic, evidence-based methodology for deciding where abstraction should be applied and where explicit control must be retained. Rather than proposing new abstractions, this work develops an empirical framework that characterizes abstraction effectiveness across multiple dimensions—performance portability, overhead attribution, tuning complexity, and robustness—enabling developers to make informed decisions about abstraction granularity.

The experimental approach leverages existing benchmark suites and application kernels, systematically measuring performance across representative workloads, abstraction layers, and hardware architectures. Through cross-layer profiling and root cause analysis, the research will build a taxonomy of abstraction failure modes and success patterns, culminating in a quantified decision framework that transforms abstraction selection from an art into an engineering discipline.

1. Research Objectives and Scope

1.1 Primary Research Questions

This research investigation seeks to answer the following fundamental questions:

1. **When does abstraction help?** Under what workload characteristics, architectural features, and use cases do high-level programming abstractions deliver on their portability promises without significant performance penalties?
2. **When does abstraction fail?** What are the systematic patterns of abstraction breakdown, and which layers (compiler, runtime, memory model) contribute to performance loss?
3. **How can we quantify the trade-off?** Can we develop measurable metrics that capture the multi-dimensional cost-benefit analysis of abstraction versus native implementation?

4. **What guidance can we provide?** Is it possible to create actionable, evidence-based decision rules that help developers choose appropriate abstraction granularity for their specific contexts?

1.2 Research Scope and Boundaries

This investigation focuses specifically on:

- **GPU-accelerated computing:** Targeting heterogeneous architectures with discrete GPU accelerators (NVIDIA, AMD, Intel)
- **Established portability frameworks:** Kokkos, RAJA, SYCL, and Julia as representative abstraction layers
- **Scientific computing kernels:** Dense and sparse linear algebra, stencil computations, and graph algorithms
- **Performance portability:** Cross-architecture performance consistency as the primary outcome metric

Explicitly out of scope:

- Proposing new programming models or runtime systems
- Distributed-memory programming (MPI layer)
- Fault tolerance and resilience mechanisms
- Power and energy consumption (though can be added as secondary metrics)

2. Experimental Design Overview

2.1 Three-Dimensional Experimental Matrix

The experimental design is structured as a three-dimensional matrix covering workload diversity, abstraction variety, and architectural heterogeneity. This systematic approach ensures comprehensive coverage of the abstraction effectiveness space.

Dimension	Categories	Justification
Workloads	Regular, Semi-irregular, Irregular	Captures spectrum from perfectly predictable to dynamically unpredictable computational patterns
Abstractions	Native, Kokkos, SYCL, RAJA, Julia	Represents dominant portability approaches in production exascale applications
Architectures	NVIDIA Ampere/Hopper, AMD CDNA2/3, Intel Ponte Vecchio	Covers three major GPU vendors with distinct memory hierarchies and programming models

2.2 Measurement Hierarchy

The investigation employs a multi-level measurement strategy, progressing from high-level performance metrics to fine-grained overhead attribution:

Level 1: Performance Metrics

- Execution time and throughput (GFLOP/s, GB/s)
- Performance Portability Coefficient (PPC) across architectures
- Efficiency relative to roofline model predictions

Level 2: Overhead Attribution

- Kernel launch latency and scheduling overhead
- Memory transfer and data movement costs
- Occupancy, utilization, and resource allocation efficiency

Level 3: Root Cause Analysis

- Compiler code generation quality (vectorization, unrolling)
- Runtime coordination and synchronization patterns
- Memory access patterns and cache behavior

Level 4: Productivity Metrics

- Source code complexity (lines of code, cyclomatic complexity)
- Tuning parameter surface area
- Time-to-performance measurements

2.3 Phased Experimental Approach

The research is structured in phases to enable incremental validation and iterative refinement:

1. **Phase 1 (Months 1-2):** Baseline establishment using native implementations
2. **Phase 2 (Months 3-4):** Abstraction layer comparative analysis
3. **Phase 3 (Months 5-6):** Deep profiling and root cause investigation
4. **Phase 4 (Months 7-8):** Taxonomy development and framework synthesis
5. **Phase 5 (Months 9-10):** Validation on real applications and tool development

3. Workload Selection and Characterization

3.1 Kernel Selection Strategy

Rather than implementing kernels from scratch, this research leverages established benchmark suites that already contain multi-implementation versions. This approach ensures reproducibility, reduces implementation bias, and enables direct comparison with existing literature.

3.2 Regular Workloads: Predictable Computation

3.2.1 STREAM Triad (Memory Bandwidth Benchmark)

Source: BabelStream (https://github.com/UoB-HPC/BabelStream)

Computational Pattern: `A[i] = B[i] + scalar * C[i]`

Characteristics:

- Perfectly regular memory access (unit stride)
- Bandwidth-bound (low arithmetic intensity ~0.25 FLOP/byte)
- No control flow divergence
- Embarrassingly parallel

Rationale: This kernel represents the best-case scenario for abstraction layers—simple, regular, and already well-optimized in most frameworks. It establishes the performance ceiling and measures baseline launch overhead.

Available Implementations: C++ (OpenMP), CUDA, HIP, SYCL, Kokkos, RAJA, OpenACC, Julia

3.2.2 Dense Matrix Multiplication (DGEMM)

Source: RAJAPerf Suite, Julia microbenchmarks (Godoy et al.)

Computational Pattern: `C = A × B` (blocked implementation)

Characteristics:

- Regular but complex memory access patterns (requires blocking for cache efficiency)
- Compute-bound (high arithmetic intensity ~16-64 FLOP/byte depending on blocking)
- Requires memory layout awareness (row-major vs. column-major)
- Sensitive to compiler optimization quality

Rationale: DGEMM tests whether abstractions can match vendor-tuned libraries and whether they properly expose memory layout choices to developers. Previous work (Godoy et al.) has shown significant variability in abstraction layer performance for this kernel.

3.2.3 3D Stencil Computation

Source: Mantevo MiniGhost, RAJAPerf

Computational Pattern: 7-point or 27-point stencil on structured grid

Characteristics:

- Regular grid traversal with halo exchanges
- Balance between memory bandwidth and compute
- Benefits from shared memory/cache blocking
- Tests multi-dimensional indexing abstractions

Rationale: Stencils are ubiquitous in scientific computing (CFD, climate, molecular dynamics) and represent a middle ground between trivial and complex kernels. They test whether abstractions can express domain decomposition and boundary conditions elegantly.

3.3 Semi-Irregular Workloads: Structured Sparsity

3.3.1 Sparse Matrix-Vector Multiplication (SpMV)

Source: RAJAPerf, Kokkos-Kernels, SuiteSparse Matrix Collection

Computational Pattern: `y = A * x` where A is sparse (CSR format)

Characteristics:

- Irregular memory access driven by sparsity pattern
- Memory-bound with poor cache locality
- Load imbalance across threads (variable row lengths)
- Performance depends on matrix structure

Rationale: SpMV is the workhorse of iterative solvers and graph algorithms. It tests whether abstractions can handle indirect memory access and whether runtime systems can mitigate load imbalance dynamically.

Matrix Selection: Test with multiple sparsity patterns:

- **Structured:** 2D/3D Laplacian (regular non-zero pattern)
- **Power-law:** Social network graph (skewed degree distribution)
- **Random:** Synthetic sparse matrix (worst-case cache behavior)

3.3.2 Sparse Triangular Solve (SpTRSV)

Source: Kokkos-Kernels, RAJA SpTRSV implementations

Computational Pattern: Forward/backward substitution on sparse triangular matrix

Characteristics:

- Inherent sequential dependencies (level-set parallelism required)
- Irregular memory access and control flow
- Requires sophisticated scheduling (e.g., coloring, level-scheduling)

Rationale: SpTRSV tests whether task-based abstractions can expose parallelism in apparently sequential code. It evaluates runtime systems' ability to discover and exploit dynamic concurrency.

3.4 Irregular Workloads: Dynamic Unpredictability

3.4.1 Breadth-First Search (BFS)

Source: Graph500 reference implementation, GAP Benchmark Suite

Computational Pattern: Level-synchronous graph traversal

Characteristics:

- Highly irregular memory access (graph topology-dependent)
- Extreme load imbalance (frontier size varies by orders of magnitude)
- Control flow divergence (different thread behaviors)
- Requires atomic operations and conflict resolution

Rationale: BFS represents the stress test for abstractions. Previous work (John et al.) shows that even task-based runtimes struggle with dynamic load balancing for irregular workloads. This kernel tests the limits of performance portability.

3.4.2 Molecular Dynamics N-Body Force Calculation

Source: CoMD (Mantevo), miniMD

Computational Pattern: Pairwise force computation with cutoff radius

Characteristics:

- Irregular neighbor lists (topology changes over time)
- Complex control flow (cutoff conditions, force accumulation)
- High arithmetic intensity but irregular memory access

Rationale: N-body codes are production workloads in materials science and biochemistry. They test whether abstractions can handle particle-based simulations with dynamic data structures.

4. Abstraction Layers and Implementations

4.1 Native Implementations (Baseline)

CUDA (NVIDIA GPUs)

Version: CUDA 12.x with nvcc compiler

Rationale: Industry-standard GPU programming model with mature toolchain and extensive optimization documentation. Serves as performance ceiling for NVIDIA hardware.

Implementation Strategy: Use hand-optimized versions from benchmark suites; ensure best practices (coalesced memory access, occupancy tuning, shared memory usage).

HIP (AMD GPUs)

Version: ROCm 5.7+ with hipcc compiler

Rationale: AMD's answer to CUDA; syntactically similar but with distinct memory hierarchy (LDS vs. shared memory, GCN/CDNA wavefront model).

Implementation Strategy: Where possible, use CUDA-to-HIP translated versions (hipify); validate performance matches native HIP implementations.

SYCL/DPC++ (Intel GPUs)

Version: Intel oneAPI 2024.x

Rationale: Intel's unified programming model for CPUs, GPUs, and FPGAs. Tests cross-vendor portability.

Note: SYCL serves dual purpose as both native implementation (for Intel) and abstraction layer (for other vendors via backends).

4.2 Kokkos: Portable C++ Abstractions

Version: Kokkos 4.x

Project URL: https://github.com/kokkos/kokkos

Backends: CUDA, HIP, SYCL, OpenMP

Key Abstraction Mechanisms

- **Execution Spaces:** Abstract hardware targets (Cuda, HIP, Serial)
- **Memory Spaces:** Explicit control over data placement (CudaSpace, HostSpace)
- **Views:** Multi-dimensional array abstraction with configurable memory layout

- **Parallel Patterns:** parallel_for, parallel_reduce, parallel_scan

Tuning Parameters Exposed

- Team size and vector length (for hierarchical parallelism)
- Memory layout (LayoutLeft vs. LayoutRight)
- Memory traits (atomic, random_access, streaming)
- Execution space configuration

Rationale for Selection

Kokkos is the most widely adopted portability layer in the Exascale Computing Project (Evans et al. report ~30% of codes use Kokkos). It represents mature abstraction technology with production-grade backends and extensive performance optimization history.

4.3 RAJA: Portable Loop Abstractions

Version: RAJA 2024.x

Project URL: https://github.com/LLNL/RAJA

Backends: CUDA, HIP, OpenMP

Key Abstraction Mechanisms

- **Execution Policies:** Templated policies specify backend and parallelism strategy
- **Index Sets:** Abstract iteration spaces (ranges, lists, segments)
- **Views:** Similar to Kokkos but with RAJA::View from camp library
- **Kernel Fusion:** Combines multiple loops into single kernel launch

Rationale for Selection

RAJA provides a different philosophical approach than Kokkos—focusing on loop-level abstractions rather than data structure abstractions. Comparison between RAJA and Kokkos reveals whether abstraction philosophy affects performance portability.

4.4 SYCL: Open Standard Heterogeneous Programming

Implementations to Test:

- **DPC++ (Intel):** Primary Intel compiler for oneAPI
- **hipSYCL/AdaptiveCpp:** SYCL implementation targeting CUDA and HIP
- **ComputeCpp (Codeplay):** Commercial SYCL implementation

Key Abstraction Mechanisms

- **Queues and Devices:** Explicit device selection and command submission

- **Buffers and Accessors:** Data management with implicit or explicit movement
- **Kernels:** Lambda-based kernel definitions with hierarchical parallelism
- **Unified Shared Memory (USM):** Pointer-based alternative to buffer model

Rationale for Selection

SYCL is an open standard (Khronos Group) aiming for true vendor neutrality. Testing multiple SYCL implementations reveals whether standard specifications translate to portable performance or whether implementation quality dominates.

4.5 Julia: High-Level Language Approach

Version: Julia 1.10+

GPU Packages:

- **CUDA.jl:** NVIDIA GPU support
- **AMDGPU.jl:** AMD GPU support (ROCm backend)
- **oneAPI.jl:** Intel GPU support (experimental)

Key Abstraction Mechanisms

- **Array Abstractions:** CuArray, ROCArray—GPU-aware arrays with automatic memory management
- **Kernel Macros:** @cuda, @roc for explicit kernel launches
- **High-Level Operations:** Broadcasting, map, reduce work across GPU arrays
- **JIT Compilation:** LLVM-based code generation for each backend

Rationale for Selection

Julia represents a radically different approach—high-level language with JIT compilation instead of C++ template metaprogramming. Godoy et al. show Julia can achieve 0.9+ efficiency on AMD but struggles on NVIDIA—investigating why is scientifically valuable.

5. Hardware Platforms and System Access

5.1 Target Architectures

Vendor	GPU Model	Architecture	Memory	Peak Performance	Access Strategy
NVIDIA	A100 SXM4	Ampere (GA100)	80GB HBM2e	19.5 TF FP64	University cluster / cloud
NVIDIA	H100 SXM5	Hopper (GH100)	80GB HBM3	60 TF FP64 (w/ sparsity)	Cloud credits / XSEDE allocation
AMD	MI250X	CDNA 2 (Aldebaran)	128GB HBM2e	47.9 TF FP64	OLCF Frontier nodes / AMD lab access
AMD	MI300A/X	CDNA 3	192GB HBM3	163 TF FP64	El Capitan early access / vendor partnership
Intel	Data Center GPU Max 1550	Ponte Vecchio	128GB HBM2e	22.2 TF FP64	ALCF Aurora nodes / Intel DevCloud

5.2 System Access Strategies

5.2.1 University/National Laboratory Resources

Primary Option: Allocations on leadership computing facilities:

- **ALCF (Aurora):** Intel GPU access through INCITE/ALCC programs
- **OLCF (Frontier):** AMD MI250X access through Director's Discretionary allocation
- **University Clusters:** Many universities now have NVIDIA A100 nodes

5.2.2 Cloud Computing Resources

AWS EC2 Instances:

- p4d.24xlarge (8x A100 80GB): $32.77/hour on-demand, ~$10-15/hour spot
- p5.48xlarge (8x H100 80GB): $98.32/hour on-demand

Microsoft Azure:

- NDm A100 v4 series: A100 80GB instances
- ND MI300X v5 series: AMD MI300X instances (preview)

Budget Estimation: Assuming 200 hours compute time per platform × 3 platforms × $15/hour average = ~$9,000 in cloud costs. Significantly reduced through academic cloud credits, spot instances, and national lab allocations.

5.2.3 Vendor Collaboration Programs

- **NVIDIA Academic Hardware Grant Program:** Potential for donated/loaned hardware
- **AMD University Research Program:** Access to AMD Accelerator Cloud
- **Intel DevCloud:** Free access to Ponte Vecchio development systems

5.3 Baseline CPU Platform

For CPU baseline comparisons and host-side profiling:

- **AMD EPYC 7763 (Milan):** 64 cores, 2.45 GHz base, commonly paired with MI250X
- **Intel Xeon Platinum 8480+:** 56 cores, 2.0 GHz base, commonly paired with Ponte Vecchio

6. Software Stack and Tools

6.1 Compilers and Programming Environments

NVIDIA Platform

Component	Version	Purpose
CUDA Toolkit	12.3+	CUDA compilation, cuBLAS/cuSPARSE libraries
NVHPC SDK	23.11+	Host compiler, math libraries
GCC	11.x or 12.x	Host C++ compiler for Kokkos/RAJA

AMD Platform

Component	Version	Purpose
ROCm	5.7+	HIP compilation, rocBLAS/rocSPARSE libraries
AOMP	Latest	AMD OpenMP compiler for CPU parallelism
hipSYCL/AdaptiveCpp	0.9.4+	SYCL implementation for AMD GPUs

Intel Platform

Component	Version	Purpose
Intel oneAPI	2024.0+	Complete oneAPI suite (DPC++, MKL, TBB)
Intel Compiler (icpx)	Latest	Host and device compilation
Level Zero	Included in oneAPI	Low-level GPU runtime API

6.2 Portability Frameworks

Framework	Version	Installation Method	Backends to Enable
Kokkos	4.2+	CMake from source	Serial, OpenMP, CUDA, HIP, SYCL
Kokkos-Kernels	4.2+	CMake (depends on Kokkos)	Same as Kokkos

RAJA	2024.02+	CMake from source or Spack	Sequential, OpenMP, CUDA, HIP
RAJA Performance Suite	Latest develop	Git clone + CMake	Matches RAJA backends
Julia	1.10+	Binary download or juliaup	N/A (language runtime)
CUDA.jl	5.x	Julia package manager	NVIDIA CUDA backend
AMDGPU.jl	0.9+	Julia package manager	AMD ROCm backend

6.3 Benchmark Suites and Applications

Benchmark Suite	URL	Languages/Models Available
BabelStream	github.com/UoB-HPC/BabelStream	C++, CUDA, HIP, SYCL, Kokkos, RAJA, Julia, OpenMP
RAJAPerf	github.com/LLNL/RAJAPerf	RAJA, CUDA, HIP, OpenMP (70+ kernels)
Mantevo Mini-Apps	mantevo.org	Multiple implementations per app
GAP Benchmark Suite	github.com/sbeamer/gapbs	C++ with OpenMP, custom CUDA versions available
SuiteSparse	sparse.tamu.edu	Matrix collection (not code, but data)

6.4 Profiling and Analysis Tools

NVIDIA Profiling Tools

Nsight Systems (nsys)

- **Purpose:** System-wide timeline profiling, kernel launch overhead analysis
- **Key Metrics:** CPU/GPU utilization, memory transfers, CUDA API calls, kernel launch latency
- **Usage Example:**

```
nsys profile --stats=true --force-overwrite=true \
--output=profile_output.qdrep ./application
```

Nsight Compute (ncu)

- **Purpose:** Detailed kernel profiling, performance counter collection

- **Key Metrics:** SM efficiency, memory throughput, occupancy, warp execution efficiency, cache hit rates
- **Usage Example:**

```
ncu --set full --target-processes all \
--kernel-name kernel_of_interest \
--launch-skip 10 --launch-count 5 \
--export report.ncu-rep ./application
```

AMD Profiling Tools

rocprof

- **Purpose:** HIP kernel profiling, performance counter collection
- **Key Metrics:** Kernel duration, memory operations, VALU/SALU utilization
- **Usage Example:**

```
rocprof --stats --hip-trace --hsa-trace \
--output-file profile.csv ./application
```

Omniperf

- **Purpose:** Comprehensive AMD GPU performance analysis (successor to rocprof)
- **Key Metrics:** Detailed memory hierarchy analysis, wavefront behavior, instruction mix
- **Usage Example:**

```
omniperf profile -n workload_name -- ./application
omniperf analyze -p workloads/workload_name/ --gui
```

Intel Profiling Tools

Intel VTune Profiler

- **Purpose:** CPU and GPU profiling for Intel architectures
- **Key Metrics:** GPU offload analysis, EU (Execution Unit) utilization, memory bandwidth

Intel Advisor

- **Purpose:** Vectorization analysis, roofline model generation

Cross-Platform Tools

LIKWID (CPU performance)

- **Purpose:** CPU performance counter access, memory bandwidth measurement
- **Tools:** likwid-perfctr, likwid-bench, likwid-topology

Linux perf

- **Purpose:** General system profiling, cache analysis
- **Usage Example:**

```
perf stat -e cycles,instructions,cache-misses,L1-dcache-loads \
-r 10 ./application
```

6.5 Data Collection and Analysis Tools

Build and Workflow Automation

- **CMake (3.20+):** Cross-platform build system
- **Spack:** HPC package manager for dependency management
- **Make/Bash scripts:** Batch experiment execution

Data Processing

- **Python 3.10+:** Primary analysis language
- **Pandas:** Tabular data manipulation and CSV parsing
- **NumPy:** Numerical computations for PPC calculation
- **Matplotlib + Seaborn:** Visualization of results
- **Jupyter Notebooks:** Interactive data exploration and reproducible analysis

Version Control and Reproducibility

- **Git:** Version control for experiment scripts and analysis code
- **Zenodo/Figshare:** Long-term data archival with DOI assignment
- **Docker/Singularity:** Containerized environments for reproducibility

7. Measurement Methodology

7.1 Performance Metrics and Definitions

Primary Performance Metric: Performance Portability Coefficient (PPC)

Following Pennycook et al. (2019), we define performance portability using the harmonic mean efficiency:

```
PPC = n / Σ(1 / Efficiency_i) where: n = number of platforms Efficiency_i =
Performance_i / Peak_i Peak_i = measured native implementation performance (not
theoretical peak)
```

Interpretation:

- PPC = 1.0: Perfect portability (equal efficiency across all platforms)
- PPC > 0.8: Excellent portability
- 0.6 < PPC < 0.8: Acceptable portability with platform-specific tuning
- PPC < 0.6: Poor portability; abstraction failing on at least one platform

Secondary Performance Metrics

Absolute Performance: Problem-specific throughput (GFLOP/s, GB/s, iterations/second)

Relative Performance: Speedup or slowdown relative to native baseline on same platform

Roofline Efficiency: Performance relative to roofline model prediction

```
Roofline_Efficiency = Achieved_GFLOP/s / min(Peak_FLOP/s, Intensity ×
Bandwidth)
```

7.2 Overhead Attribution Methodology

Level 2.1: Kernel Launch Overhead

Measurement Protocol:

1. Measure total application time with high-resolution timers (std::chrono or CUDA events)
2. Extract kernel execution time from profiler (pure GPU compute time)
3. Calculate overhead: `Overhead = Total_Time - Kernel_Time`
4. Decompose into:
    - Launch latency (time between host API call and kernel start)

- Synchronization overhead (explicit or implicit barriers)
- Host-side framework overhead (parallel pattern setup)

Tool Usage: nsys for NVIDIA (CUDA API timeline), rocprof --hip-trace for AMD

Level 2.2: Memory Transfer Overhead

What to Measure:

- Explicit memory copies (cudaMemcpy/hipMemcpy)
- Implicit transfers (unified memory page faults)
- Framework-induced extra copies (abstraction layer data marshaling)

Detection Method: Profile memory transfer traces; compare total data movement between native and abstraction versions

Level 2.3: Resource Utilization

Metric	NVIDIA (ncu)	AMD (omniperf)	Interpretation
Occupancy	sm__warps_active.avg	Wavefront occupancy	Utilization of GPU compute resources; <70% suggests underutilization
SM/CU Efficiency	smsp__cycles_active.avg	VALU busy %	Fraction of time execution units are doing useful work
Memory Throughput	dram__bytes.sum.per_second	Memory bandwidth achieved	Compare to peak bandwidth; <50% suggests non-optimal access patterns
L2 Cache Hit Rate	lts__t_sector_hit_rate	L2 cache hit %	Data reuse effectiveness; abstraction may hurt spatial locality

7.3 Root Cause Analysis Protocol

Step 1: Identify Performance Gap

```
IF Abstraction_Performance < 0.85 × Native_Performance THEN Proceed to Step 2
(significant gap warrants investigation)
```

Step 2: Profile Both Implementations

- Collect full performance counter sets for native and abstraction versions
- Generate side-by-side comparison tables

Step 3: Inspect Generated Code

For Kokkos/RAJA (C++ template-based):

```
# Generate assembly or PTX/GCN code nvcc -ptx -lineinfo -O3 kokkos_kernel.cu -o
kokkos_kernel.ptx hipcc --save-temps -O3 kokkos_kernel.cpp # Compare with
native version diff native_kernel.ptx kokkos_kernel.ptx
```

For Julia:

```
using CUDA @device_code_ptx @cuda threads=256 my_kernel(args...) # Inspect
generated PTX; look for suboptimal patterns
```

Step 4: Attribute to Layer

Categorize root cause into taxonomy:

- **Compiler Backend Failure:** Poor code generation (missed optimizations, bad unrolling)
- **Runtime Coordination Overhead:** Extra synchronization, task scheduling costs
- **Memory Model Mismatch:** Abstraction forces non-optimal layout or extra indirection
- **API Limitation:** Abstraction cannot express optimization (e.g., shared memory tiling)

7.4 Tuning Complexity Measurement

Quantitative Metrics

1. Tuning Parameter Count

For each implementation, count exposed parameters requiring human decision:

```
Native CUDA DGEMM: - Block dimensions (blockDim.x, blockDim.y): 2 params - Tile
size: 1 param - Shared memory config: 1 param - Stream concurrency: 1 param
Total: 5 parameters Kokkos DGEMM: - Team size: 1 param - Vector length: 1 param
- Tile size: 1 param - Execution space config: 1 param - Memory layout
(LayoutLeft/Right): 1 param - Memory traits: 1-3 params (optional) Total: 6-8
parameters
```

2. Lines of Code (LoC)

Measure kernel implementation size; abstractions should reduce LoC but may increase due to boilerplate

3. Time-to-Performance

Simulate tuning process:

1. Start with default/naive configuration
2. Measure performance

3. Apply standard optimizations iteratively
4. Record time (in tuning iterations) to reach 80% of best-known performance

Qualitative Assessment

- **Error Message Quality:** Rate clarity of compiler/runtime errors (1-5 scale)
- **Debugging Difficulty:** Can you set breakpoints in actual kernel code? Or is it buried in generated code?
- **Documentation Completeness:** Are tuning guidelines available and accurate?

7.5 Experimental Controls and Statistical Rigor

Timing Methodology

- **Warm-up:** Execute kernel 10 times before measurement to amortize compilation overhead (JIT) and warm caches
- **Repetitions:** Measure 30 executions per configuration
- **Reporting:** Report median (robust to outliers) and interquartile range (IQR)
- **System State:** Disable CPU frequency scaling (set to performance governor), disable GPU boost clocks for reproducibility

Problem Sizes

Test multiple problem sizes to capture different performance regimes:

- **Small:** Fits in L2 cache; exposes launch overhead
- **Medium:** Fits in GPU memory; balanced compute/memory
- **Large:** Stresses memory bandwidth; may expose scaling issues

Example for DGEMM:

- Small: N=1024 (8 MB per matrix)
- Medium: N=4096 (128 MB per matrix)
- Large: N=16384 (2 GB per matrix)

8. Data Collection and Management

8.1 Experiment Automation Framework

Directory Structure

```
measurement-driven-abstraction/ ├── benchmarks/ # Source code for each benchmark │ ├── stream/ │ ├── dgemm/ │ ├── spmv/ │ └── bfs/ ├── scripts/ # Automation scripts │ ├── build_all.sh # Compile all implementations │ ├── run_experiments.py # Batch execution │ └── collect_profiles.sh # Profile collection ├── results/ # Raw experiment output │ ├── nvidia_a100/ │ ├── amd_mi250x/ │ └── intel_pvc/ ├── analysis/ # Data processing and visualization │ ├── parse_results.py │ ├── compute_ppc.py │ ├── generate_plots.py │ └── notebooks/ ├── data/ # Structured data storage │ ├── performance.csv │ ├── overhead.csv │ └── profiling_metrics.csv └── paper/ # Manuscript and figures
```

8.2 Automated Experiment Execution

Master Experiment Driver (Python)

```
# run_experiments.py (pseudo-code structure) KERNELS = ["stream", "dgemm", "spmv", "bfs"] ABSTRACTIONS = ["native", "kokkos", "raja", "sycl", "julia"] PLATFORMS = ["nvidia_a100", "amd_mi250x", "intel_pvc"] PROBLEM_SIZES = ["small", "medium", "large"] for kernel in KERNELS: for abstraction in ABSTRACTIONS: for platform in PLATFORMS: for size in PROBLEM_SIZES: # Check if implementation exists for this combination if implementation_available(kernel, abstraction, platform): # Run performance test perf_data = run_benchmark(kernel, abstraction, platform, size, iterations=30) save_results(perf_data, "performance.csv") # Run profiling (subset of runs to save time) if should_profile(kernel, abstraction): profile_data = run_profiler(kernel, abstraction, platform, size) save_results(profile_data, "profiling_metrics.csv")
```

8.3 Data Schema and Storage

Performance Results Table (CSV)

Column	Type	Description
timestamp	datetime	When experiment was run
kernel	string	Benchmark name (stream, dgemm, etc.)
abstraction	string	Implementation type
platform	string	Hardware identifier

problem_size	integer	Problem dimension (N for DGEMM, array size for STREAM, etc.)
execution_time_ms	float	Median execution time over 30 runs
throughput	float	Performance in problem-specific units (GFLOP/s, GB/s)
efficiency	float	Performance relative to native baseline

Profiling Metrics Table (CSV)

Column	Type	Description
experiment_id	string	Foreign key to performance table
kernel_time_ms	float	Pure GPU kernel execution time
launch_overhead_ms	float	Time spent in kernel launch and setup
memory_transfer_mb	float	Total data transferred host↔device
occupancy_percent	float	GPU occupancy
sm_efficiency_percent	float	Streaming multiprocessor utilization
memory_bandwidth_achieved_gbs	float	Measured memory throughput
l2_cache_hit_rate	float	L2 cache hit percentage

8.4 Analysis Scripts

Performance Portability Coefficient Calculation

```
# compute_ppc.py (Python with Pandas) import pandas as pd import numpy as np
def calculate_ppc(df, abstraction, kernel, size): """Calculate PPC for given
abstraction across all platforms""" # Get native performance on each platform
(baseline) native = df[(df['abstraction'] == 'native') & (df['kernel'] ==
kernel) & (df['problem_size'] == size)] # Get abstraction performance abstract
= df[(df['abstraction'] == abstraction) & (df['kernel'] == kernel) &
(df['problem_size'] == size)] # Calculate efficiency on each platform
efficiencies = [] for platform in abstract['platform'].unique(): native_perf =
native[native['platform'] == platform]['throughput'].values[0] abstract_perf =
abstract[abstract['platform'] == platform]['throughput'].values[0] efficiency =
abstract_perf / native_perf efficiencies.append(efficiency) # PPC = harmonic
mean of efficiencies n = len(efficiencies) ppc = n / sum(1/e for e in
efficiencies) return ppc, efficiencies # Example usage df = pd.read_csv('data/
performance.csv') ppc, effs = calculate_ppc(df, 'kokkos', 'dgemm', 4096)
print(f"Kokkos DGEMM (N=4096) PPC: {ppc:.3f}") print(f"Per-platform
efficiencies: {effs}")
```

9. Taxonomy Development and Framework Synthesis

9.1 Failure Mode Taxonomy Construction

As performance gaps are identified and root causes investigated, we categorize patterns into a structured taxonomy. This taxonomy becomes the knowledge base for the decision framework.

Taxonomy Structure

Pattern Template:

Pattern Name: Descriptive identifier
Symptom: Observable performance characteristic
Root Cause: Technical explanation of failure mechanism
Affected Workloads: Computational characteristics that trigger this pattern
Affected Platforms: Architecture-specific or cross-platform
Mitigation Strategies: Actionable recommendations
Evidence: Reference to experimental data supporting this pattern

Example Taxonomy Entries (Hypothetical Based on Literature)

Pattern 1: Launch Overhead Dominance
Symptom: Abstraction 30-60% slower than native for kernels with execution time <50 μs
Root Cause: Extra function call layers in abstraction framework not fully inlined by compiler; runtime dispatch overhead
Affected Workloads: Fine-grained kernels, irregular applications with many small kernel launches
Affected Platforms: All (cross-platform pattern)
Mitigation: Use native implementation for inner-loop hotspots, OR coarsen task granularity, OR enable aggressive inlining flags
Evidence: Observed in RAJA SpMV on all platforms; nsys profiling shows 45% time in CUDA API calls vs. 12% for native

Pattern 2: Compiler Backend Unrolling Failure
Symptom: Abstraction 2-4× slower on NVIDIA for nested loops; performance matches on AMD
Root Cause: PTX backend in nvcc fails to unroll complex loop nests generated by abstraction layer; AMD compiler handles same pattern correctly
Affected Workloads: Multi-dimensional array access, complex indexing expressions
Affected Platforms: NVIDIA-specific (PTX code generation issue)

Mitigation: Manual loop unrolling in source, OR use vendor-tuned library (cuBLAS), OR file compiler bug report
Evidence: Julia DGEMM on A100 (Godoy et al. 2023); PTX inspection shows no unrolling; 40% performance penalty in single precision

Pattern 3: Memory Layout Mismatch
Symptom: 20-40% bandwidth reduction compared to native
Root Cause: Abstraction layer defaults to non-optimal memory layout (e.g., column-major on row-major architecture); results in non-coalesced access
Affected Workloads: Multi-dimensional arrays with high data reuse
Affected Platforms: Platform-dependent (NVIDIA prefers LayoutLeft, AMD more flexible)
Mitigation: Explicit layout specification in abstraction API (increases tuning burden)
Evidence: Kokkos stencil benchmark; L2 cache hit rate drops from 75% (native) to 45% (Kokkos default layout)

9.2 Success Pattern Characterization

Equally important: document when abstractions work well

Success Pattern 1: Regular Dense Computation
Observation: Kokkos achieves PPC > 0.90 for STREAM triad across all platforms
Explanation: Simple memory access pattern + high compute intensity = compiler can optimize effectively
Generalization: Abstractions excel when workload matches assumptions (regular parallelism, predictable memory access)
Actionable Insight: For bandwidth-bound regular kernels, abstraction overhead is negligible; choose portability

9.3 Decision Framework Construction

Synthesize taxonomy into quantified decision rules.

Rule-Based Decision Logic (Pseudo-code)

```
function recommend_abstraction_strategy(workload_profile, target_platforms): #
Extract workload characteristics (from profiling or static analysis) regularity
= measure_memory_regularity(workload) # 0-1 scale compute_intensity =
calculate_arithmetic_intensity(workload) # FLOP/byte kernel_duration =
estimate_kernel_time(workload) # microseconds control_divergence =
analyze_branch_behavior(workload) # 0-1 scale # Query empirical database for
expected PPC expected_ppc = lookup_ppc_from_taxonomy(workload_profile,
abstraction, platforms) # Decision logic based on thresholds derived from
```

```
experiments if regularity > 0.8 and control_divergence < 0.1: # Regular workload: abstractions likely to succeed if kernel_duration > 100: # Sufficient time to amortize launch overhead if expected_ppc > 0.80: return "FULL_ABSTRACTION: Use Kokkos/RAJA/SYCL" elif expected_ppc > 0.60: return "ABSTRACTION_WITH_TUNING: Use framework but expect platform-specific tuning" else: return "CAUTION: Kernel too short; launch overhead may dominate. Profile before committing." elif regularity > 0.5 and control_divergence < 0.3: # Semi-irregular: selective abstraction # Use abstraction for host orchestration, native for GPU kernels return "HYBRID: Kokkos for data management + native CUDA/HIP kernels for hot loops" else: # Irregular workload: abstractions likely to fail if supports_task_runtime(workload): return "TASK_RUNTIME: Consider PaRSEC/Legion if load balancing is critical" else: return "NATIVE: Use platform-specific implementation with expert tuning" # Cross-platform variance check ppc_variance = calculate_ppc_variance(expected_ppc, platforms) if ppc_variance > 0.3: return "WARNING: High cross-platform variance. Consider platform-specific branches." return recommendation
```

Quantified Thresholds (To Be Determined from Experiments)

Threshold Name	Hypothetical Value	How to Determine Empirically
Memory regularity cutoff	0.8	Cluster analysis of stride variance in successful vs. failed cases
Minimum kernel duration	100 µs	Plot overhead fraction vs. kernel time; find inflection point
PPC success threshold	0.80	Survey of production HPC codes: what PPC do they tolerate?
Control divergence limit	0.3	Correlation analysis: PPC vs. warp divergence from profilers

> **Note:** These thresholds are *hypotheses* to be validated/refined during the experimental phase. The key contribution is the *methodology* for determining such thresholds empirically rather than guessing them.

10. Validation and Tool Development

10.1 Retrospective Validation

Objective: Test whether the decision framework would have predicted actual abstraction choices made in production exascale applications.

Validation Methodology

1. **Select Case Studies:** Choose 5-8 applications from Evans et al.'s 62 ECP codes that are open-source and have both native and abstraction implementations
2. **Profile Workload Characteristics:** Run profiling tools on their hotspot kernels to extract regularity, compute intensity, etc.
3. **Apply Decision Framework:** Feed workload profiles into decision logic; record recommendation
4. **Compare to Reality:** Did application developers make the same choice? If not, why?
5. **Analyze Discrepancies:** Refine framework based on real-world constraints not captured in initial model

Example Case Studies

- **ExaSMR (reactor simulation):** Uses Kokkos extensively; expect framework to recommend full abstraction for regular stencils
- **QMCPACK (quantum Monte Carlo):** Uses CUDA+OpenMP hybrid; expect framework to recommend selective abstraction due to irregular sampling
- **LAMMPS (molecular dynamics):** Has both Kokkos and native CUDA versions; compare performance predictions

10.2 Prospective Validation

Objective: Work with application teams to apply framework during development and measure outcomes.

Validation Protocol

1. **Recruit Collaborators:** Engage with 2-3 application teams actively developing new codes or porting to new architectures
2. **Baseline Phase:** Profile their current implementation (if exists) or prototype
3. **Recommendation Phase:** Apply decision framework; provide abstraction strategy recommendation
4. **Implementation Phase:** Team implements following recommendations (with researcher support)

5. **Measurement Phase:** Measure outcomes:
    - Time-to-first-performance: How long to get working code?
    - Time-to-target-performance: How long to reach 80% of native?
    - Final PPC: Did portability goals materialize?
    - Developer satisfaction: Qualitative feedback on process

10.3 Decision Support Tool Development

Tool Architecture

Component 1: Workload Profiler

- Lightweight instrumentation that extracts workload characteristics
- Integrates with existing profilers (nsys, rocprof)
- Outputs: regularity score, compute intensity, kernel duration distribution

Component 2: Taxonomy Database

- Structured storage of failure/success patterns from experiments
- Queryable by workload characteristics and target platforms
- Returns: expected PPC, known issues, mitigation strategies

Component 3: Recommendation Engine

- Implements decision logic from Section 9.3
- Ingests workload profile + target platforms
- Outputs: abstraction strategy with confidence level and rationale

Example Tool Usage (Command-Line Interface)

```
# Profile an application $ abstraction-advisor profile --app ./my_simulation --
args "input.dat" Profiling application... Detected 3 major kernels: 1.
sparse_matvec: 45% of runtime, irregular (regularity=0.35) 2. dense_gemm: 30%
of runtime, regular (regularity=0.92) 3. stencil_update: 25% of runtime, semi-
regular (regularity=0.68) Workload profile saved to: workload_profile.json #
Get recommendation $ abstraction-advisor recommend --profile
workload_profile.json \ --targets nvidia_a100,amd_mi250x --priority portability
Analyzing workload against empirical database... RECOMMENDATION: Hybrid
Abstraction Strategy Kernel 1 (sparse_matvec): Strategy: NATIVE implementation
(CUDA/HIP) Rationale: High irregularity (0.35) + known abstraction failures for
SpMV Expected PPC with Kokkos: 0.42 (poor portability) Expected PPC with
native: 0.88 (separate tuned versions) Kernel 2 (dense_gemm): Strategy:
ABSTRACTION (Kokkos or vendor library) Rationale: Regular workload (0.92) +
high compute intensity Expected PPC with Kokkos: 0.89 (excellent portability)
Alternative: Use cuBLAS/rocBLAS (PPC: 0.95) Kernel 3 (stencil_update):
Strategy: ABSTRACTION with platform-specific tuning Rationale: Semi-regular
(0.68); abstractions work but require tuning Expected PPC with Kokkos: 0.76
(acceptable with tuning) Tuning parameters to focus on: memory layout, tile
```

```
size OVERALL STRATEGY: - Use Kokkos for host orchestration and data management
- Implement Kernel 1 in native CUDA/HIP (use backend-specific macros) - Use
Kokkos for Kernel 2 and 3 - Budget 2-3 days for platform-specific tuning of
Kernel 3 Confidence: MEDIUM (based on 45 similar workloads in database)
```

10.4 Artifact Release and Community Engagement

Open-Source Components:

- Decision support tool (GitHub repository with BSD/MIT license)
- Complete experimental dataset (Zenodo with DOI)
- Reproducibility package (containerized environments, scripts)
- Interactive taxonomy browser (web interface for exploring patterns)

Community Contribution Pathway:

- Accept pull requests for new workload characterizations
- Crowdsource additional failure/success patterns
- Maintain living database that grows with community contributions

11. Timeline and Milestones

11.1 10-Month Research Plan

Month	Phase	Activities	Deliverables
1	Setup	Secure platform access, install software stack, clone benchmark suites	Working environments on all platforms
2	Baseline	Run native implementations, establish performance ceilings	Baseline performance database
3-4	Abstraction Comparison	Run Kokkos, RAJA, SYCL, Julia versions; collect performance data	Complete performance dataset (300+ experiments)
5	Deep Profiling	Profile all significant performance gaps with vendor tools	Overhead attribution tables
6	Root Cause Analysis	Inspect generated code, identify failure mechanisms	Initial taxonomy (8-12 patterns documented)
7	Framework Development	Synthesize taxonomy into decision rules, determine thresholds	Decision framework prototype
8	Tool Development	Implement decision support tool, validate on held-out kernels	Working prototype tool
9	Validation	Retrospective validation on ECP codes, prospective case study	Validation results
10	Dissemination	Write paper, prepare artifact release, create documentation	Manuscript submission, artifact package

11.2 Critical Path and Risk Mitigation

Risk 1: Platform Access Delays

Mitigation: Apply for allocations early (3+ months before needed); use cloud resources as backup; prioritize NVIDIA platform (easiest access) for initial results

Risk 2: Benchmark Implementation Gaps

Mitigation: Pre-survey available implementations before finalizing kernel list; be willing to drop abstraction-kernel combinations that don't have existing code

Risk 3: Profiling Tool Issues

Mitigation: Test profiling tools early in Month 1; establish workarounds (e.g., manual instrumentation) if vendor tools fail

Risk 4: Too Much Data

Mitigation: Focus on minimum viable matrix first (3 kernels × 3 abstractions × 2 platforms = 18 experiments); expand later if time permits

12. Expected Outcomes and Impact

12.1 Scientific Contributions

1. **Empirical Taxonomy:** First systematic classification of abstraction failure modes and success patterns across architectures and workloads
2. **Measurement Methodology:** Reproducible protocol for cross-layer performance attribution in abstraction layers
3. **Quantified Decision Framework:** Evidence-based thresholds and rules for abstraction selection, replacing heuristics with data
4. **Performance Portability Dataset:** Comprehensive benchmark results covering 5+ abstractions × 3 architectures × 8 workloads = 120+ configurations

12.2 Practical Impact

- **For Application Developers:** Actionable guidance on when to use abstractions vs. native code; reduces trial-and-error
- **For Framework Developers:** Evidence identifying which failure modes to prioritize fixing; competitive analysis
- **For Tool Builders:** Foundations for compiler/runtime systems that adapt abstraction granularity automatically
- **For Funding Agencies:** Objective assessment of portability framework maturity; inform investment decisions

12.3 Publication Strategy

Primary Venue: Special Issue submission (tailored to venue requirements)

Supplementary Publications:

- Tool paper at HPC conference (SC, ISC, PPoPP)
- Extended dataset paper at journal (Scientific Data, TPDS)
- Position paper on future of performance portability (CACM, IEEE Computer)

12.4 Long-Term Research Directions

This work establishes foundations for:

- **Automated Abstraction Selection:** Compilers that choose abstraction level based on workload analysis

- **Adaptive Runtime Systems:** Runtimes that switch between abstraction layers dynamically based on profiling
- **Abstraction Co-Design:** Informing next-generation programming model design with empirical evidence
- **Cross-Domain Extension:** Applying methodology to other domains (AI/ML frameworks, cloud computing)

13. Resource Requirements Summary

13.1 Computational Resources

- ~500 GPU-hours across NVIDIA, AMD, Intel platforms
- Priority: secure allocations on national lab systems (ALCF, OLCF)
- Backup: cloud computing budget $8,000-$12,000 (seek academic credits)

13.2 Software and Licensing

- All core software is open-source (Kokkos, RAJA, SYCL implementations)
- Vendor compilers and profilers are freely available for academic use
- No commercial software licenses required

13.3 Human Resources

- Primary researcher: Full-time for 10 months
- Advisor/senior collaborator: Guidance on experimental design, paper writing
- Potential undergraduate assistant: Help with data collection automation (optional)
- Collaborating application teams: 2-3 teams for validation phase

14. Conclusion

This experimental design provides a comprehensive, systematic approach to addressing the critical gap in exascale programming: the lack of measurement-driven methodology for selective abstraction. By combining rigorous empirical measurement, cross-layer performance attribution, and systematic taxonomy construction, this research will transform abstraction selection from an art based on experience and intuition into an engineering discipline grounded in evidence.

The phased approach—from baseline establishment through deep profiling to framework synthesis—ensures incremental validation and allows for iterative refinement. The use of established benchmark suites and real application kernels maximizes reproducibility and practical relevance while minimizing

implementation risk. The resulting decision framework and supporting tools will provide immediate value to the HPC community while establishing foundations for next-generation adaptive systems.

Most importantly, this work does not propose yet another abstraction layer or programming model. Instead, it provides the missing measurement infrastructure and empirical evidence base that the community needs to make informed decisions about existing abstractions. This meta-level contribution —teaching the community how to evaluate abstractions systematically—has the potential for broader and more lasting impact than any single programming model innovation.

End of Document